SAAS TECHNO-FUNCTIONAL PROPOSAL

Autonomous AI HR Interviewer & Proctoring Suite

A highly articulated, complete systems specification and structural design blueprint for a decoupled multi-tenant cloud application layer. Fully optimized to connect with Odoo Recruitment via secure Webhooks and external REST APIs to provide natural voice screening combined with zero-footprint proctoring.

1. Executive Vision & Value Proposition

1.1 The Operational Crises of Scale in Modern Recruitment

In the modern corporate ecosystem, human capital acquisition remains the single largest operational cost center and, simultaneously, the most critical engine of organic competitive differentiation. However, early-stage talent acquisition funnels are broken by systematic inefficiencies. Human Resource departments and technical screening teams routinely face high-volume applicant pipelines that degrade review consistency and introduce immense operational drag.

The traditional approach requires human recruiters to manually evaluate hundreds of incoming resumes, organize phone-screening touchpoints, and administer unstandardized initial evaluations. This human-dependent process introduces significant cognitive fatigue, leading directly to high-quality candidates slipping through loose filters while unqualified candidates advance to expensive, late-stage technical panels. The resultant metric of organization-wide friction is an unsustainable Mean Time to Hire (MTTH) and high administrative overhead, limiting organizational flexibility.

1.2 The Systemic Defeat of Written Screening in the Era of Generative AI

To bypass human screening resource limits, enterprise organizations have turned over the past decade to unmonitored written screening systems, third-party coding sandbox tools, and asynchronous multiple-choice assessment services. This methodology is now completely defeated. The rapid emergence and democratization of consumer-grade Generative Artificial Intelligence (AI) models—such as GPT-4 and Claude—have permanently decoupled written evaluation scores from actual candidate proficiency.

Applicants routinely utilize secondary screens, split browsers, and system-level transcription extensions to copy-paste assessment text and fetch optimal, syntactically flawless answers in real-time. Unproctored written assessments are no longer a measurement of organic cognitive reasoning or technical competence. Instead, they act as an evaluation of a candidate's speed in operating real-time generative interfaces. As a direct consequence, recruiting pipelines are flooded with candidates who score perfectly on pre-screens but fail completely during live, on-site execution environments.

1.3 The Value Proposition of Autonomous Conversational Screening

The proposed SaaS platform acts as a secure, automated screening layer designed to restore integrity and efficiency to the hiring funnel. By orchestrating bidirectional, low-latency vocal assessments, the platform bypasses the vulnerabilities of written tests. Rather than utilizing static, predictable question lists, the system operates as a dynamic conversationalist. It reads, analyzes, and understands spoken answers in real-time, instantly

formulating intelligent, contextually relevant follow-up questions to probe the candidate's actual depth of understanding.

Simultaneously, the platform implements a zero-footprint proctoring engine running purely within standard browser frameworks. Candidates do not need to install invasive operating system-level monitoring software or browser plugins. Instead, native Web APIs are configured to securely capture and analyze eye movements, facial orientation anomalies, and browser-focus focus changes on-the-fly. The compiled assessment data is aggregated into a high-fidelity trust profile, giving hiring managers verified talent metrics long before a human recruiter conducts their first face-to-face meeting.

> **THE STANDARDIZED EVALUATION STANDARD**
>
> By presenting identical baseline challenges to every candidate within a target vacancy and grading responses using structured evaluation criteria, the system removes unconscious human bias, establishing a highly reliable and legally compliant hiring standard.

1.4 Strategic Advantages of a Decoupled Cloud Architecture

Building the SaaS platform as a decoupled, external cloud layer rather than a localized Odoo custom module guarantees maximum scalability and platform compatibility. Odoo modules are structurally bound to the Python ERP runtime environment, native database schemas, and rigid versioning schedules.

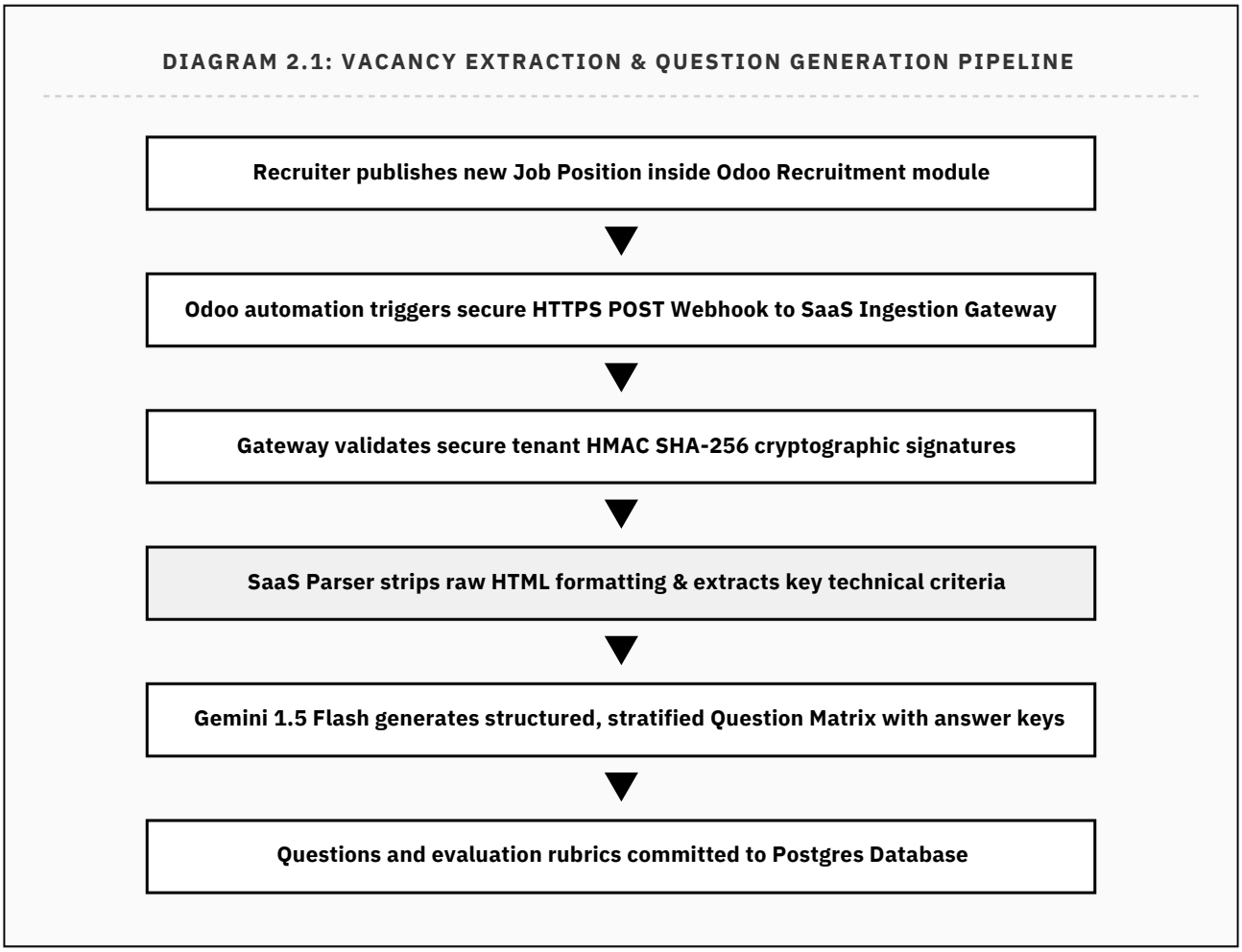
By isolating the SaaS backend core as an independent, API-first service, several key operational advantages are achieved:

- **Broad Environment Compatibility:** The SaaS integrates with Odoo Online (SaaS, which strictly forbids the installation of custom backend Python packages), Odoo.sh, and self-hosted On-Premise deployments with zero code modifications.
- **Mitigating Version-Lock Risk:** The system is fully insulated from major Odoo core migrations (e.g., v17 to v18). The interface remains completely stable as long as Odoo's core REST or XML-RPC APIs remain backward compatible.
- **Optimized Resource Allocation:** Processing real-time, low-latency audio pipelines, managing machine learning inference loops, and analyzing webcam biometric telemetry are CPU-intensive processes. Running these tasks on an independent, auto-scaling cloud cluster protects the client's critical ERP system from resource starvation.

2. Granular Functional Workflows

2.1 Phase A: Vacancy Ingestion & Evaluation Matrix Construction

The system's operational loop is initiated automatically when a new vacancy is published inside Odoo. The platform's goal is to parse natural-language job descriptions, extract functional requirements, and generate a structured, stratified interview matrix.



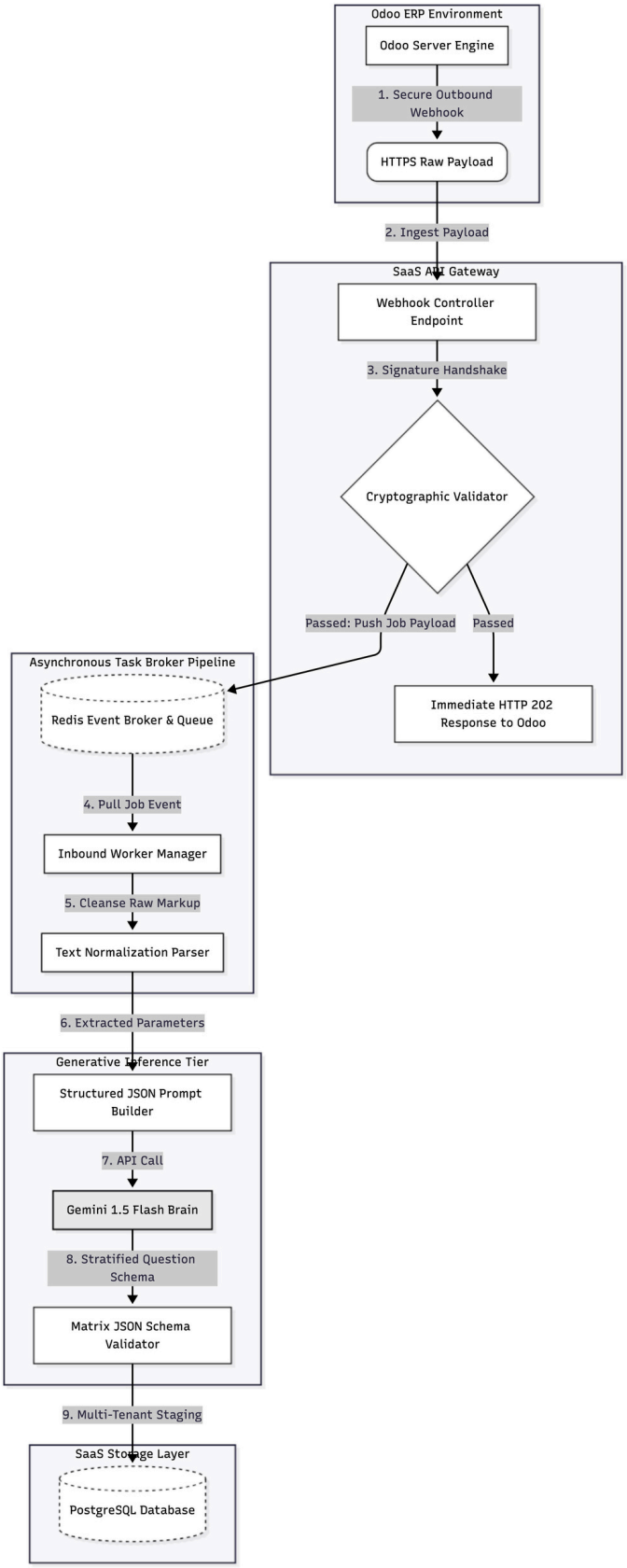
When a recruiter updates a job state to "Active" or "Published" inside the Odoo ATS, an outbound webhook payload is generated containing the Job ID, title, and the unformatted text block of the job description. The SaaS Integration Gateway receives the POST request, verifies the cryptographically signed headers, and parses the text into core skill nodes.

These skill nodes are fed to Google's Gemini 1.5 Flash API with strict system instructions to compile a structured, multi-difficulty question matrix. The model is forced to output exactly four distinct difficulty levels:

1. **Basic:** High-level definitions, standard terminologies, and framework conventions.

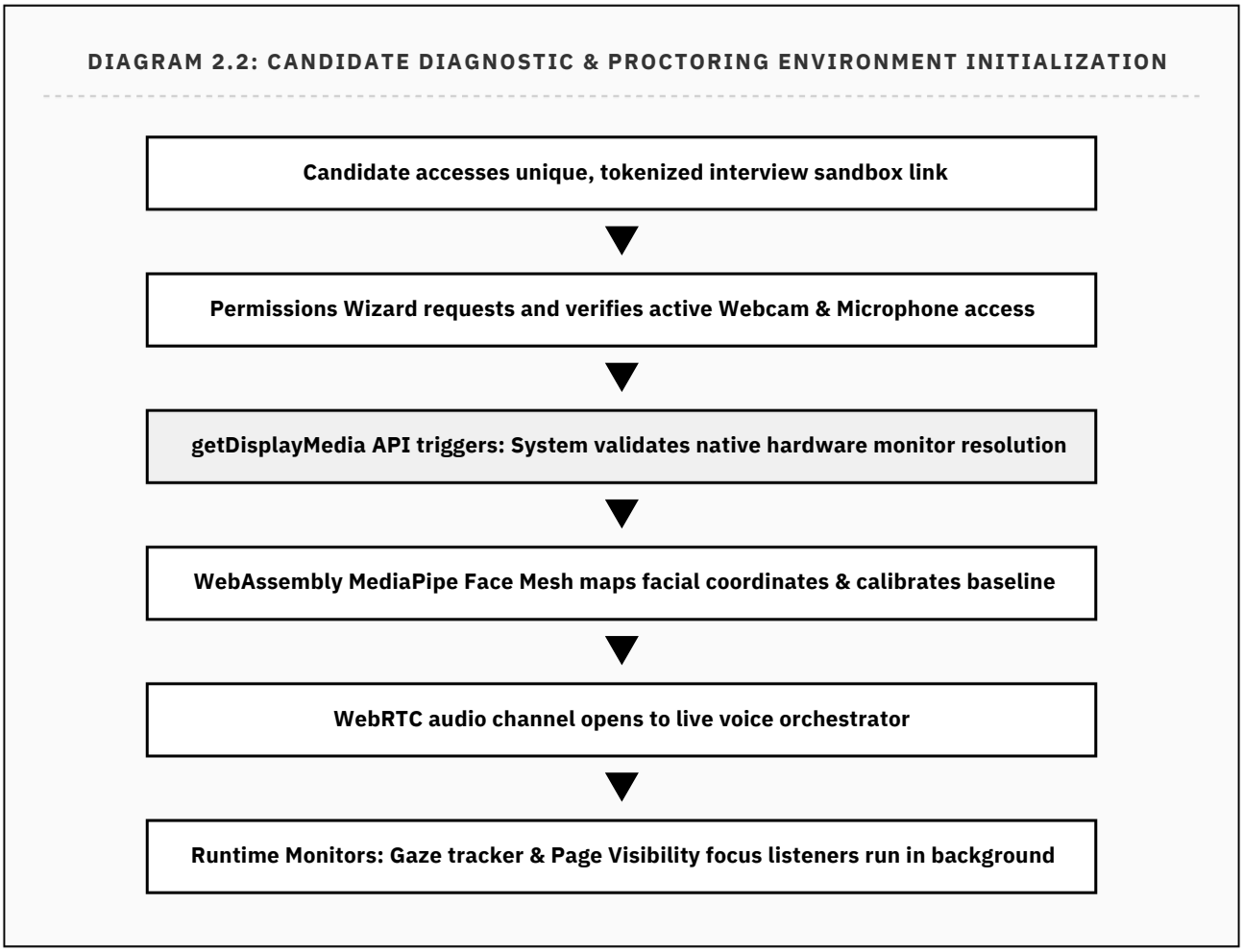
2. **Easy:** Simple syntactic configurations, standard commands, and basic troubleshooting steps.
3. **Medium:** Systems design trade-offs, optimization practices, and operational logic routing.
4. **Hard:** High-concurrency failures, edge-case troubleshooting, and disaster recovery strategies.

The generated question sets are committed to PostgreSQL, indexed directly under the tenant ID and Odoo Job ID, establishing the structural framework for subsequent applicant sessions.



2.2 Phase B: Identity Verification, Device Diagnostics & Active Proctoring

When a candidate accesses the screening workspace, the SaaS platform locks down the client-side browser space, verifies the integrity of the local hardware, and initializes active proctoring trackers.

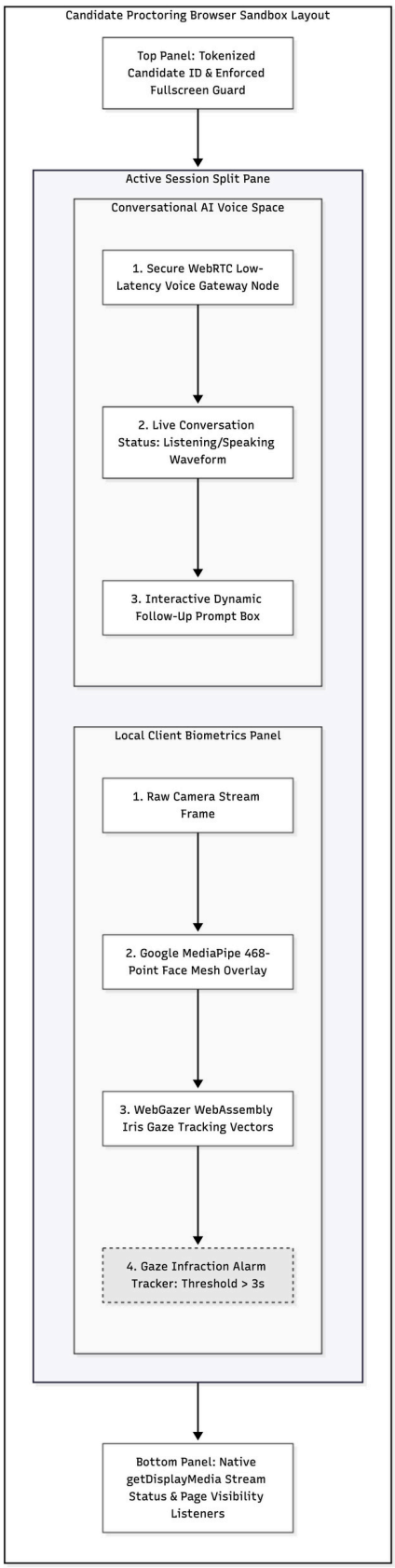


When the candidate clicks their scheduled single-use link, they are taken to a Next.js testing sandbox application. The interface initiates a diagnostic wizard requiring hardware access permissions. Both webcam and microphone access are mandatory; refusing permissions immediately blocks the candidate from proceeding.

Following successful hardware checks, the system starts the **Display Stream Acquisition Handshake**. The browser triggers a `getDisplayMedia()` prompt. The SaaS backend analyzes the incoming video track parameters:

- If the stream width and height do not match the system's screen resolution (indicating a shared tab or window), the interface blocks candidate progression and requires a full-screen share.
- This ensures that any secondary screens, open files, chat channels, or local AI applications are fully visible to the proctoring stream.

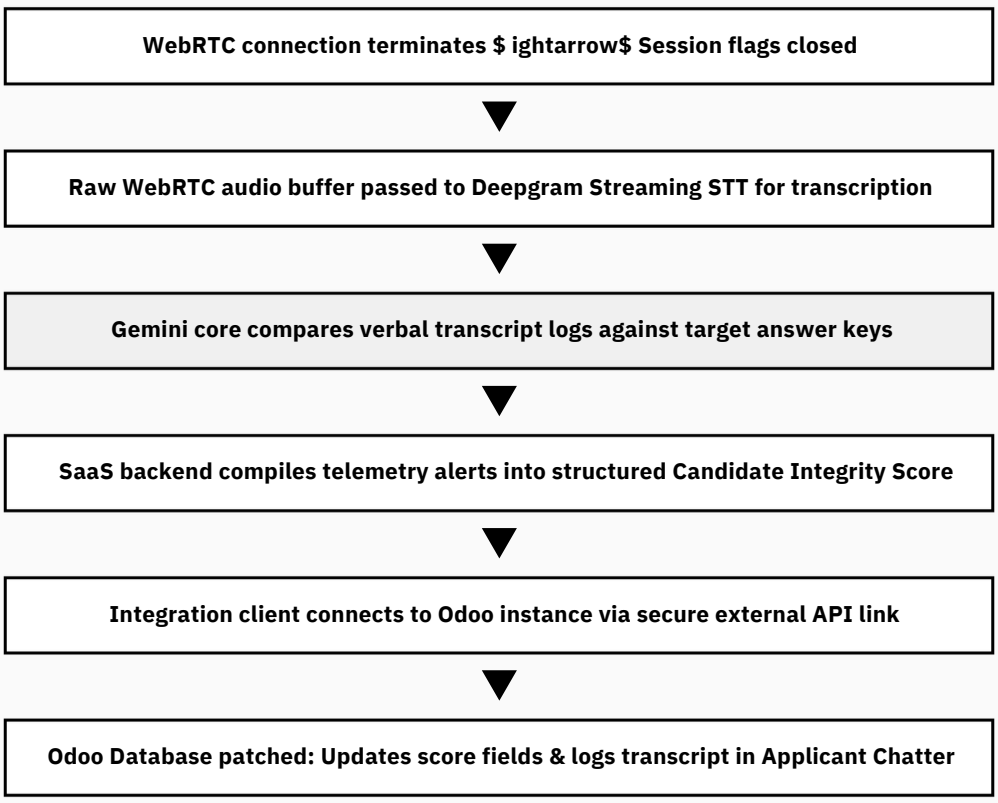
The system then initializes a client-side face mesh (via WebAssembly-compiled Google MediaPipe). The webcam maps 468 distinct landmark coordinates across the candidate's facial structure, tracking the baseline gaze vectors. Once calibration is complete, a bi-directional WebRTC connection opens. Throughout the live call, Page Visibility listeners log tab switches, while the local face mesh flags visual gaze deviations lasting longer than 3 consecutive seconds.



2.3 Phase C: Session Breakdown, Scoring Analysis & Odoo Integration Loop

The moment the candidate terminates the session, the SaaS backend moves processing to asynchronous worker queues to compile transcripts and sync scores directly to Odoo.

DIAGRAM 2.3: SESSION TEARDOWN, ANALYTICAL ASSESSMENT, & ODOO SYNCHRONIZATION



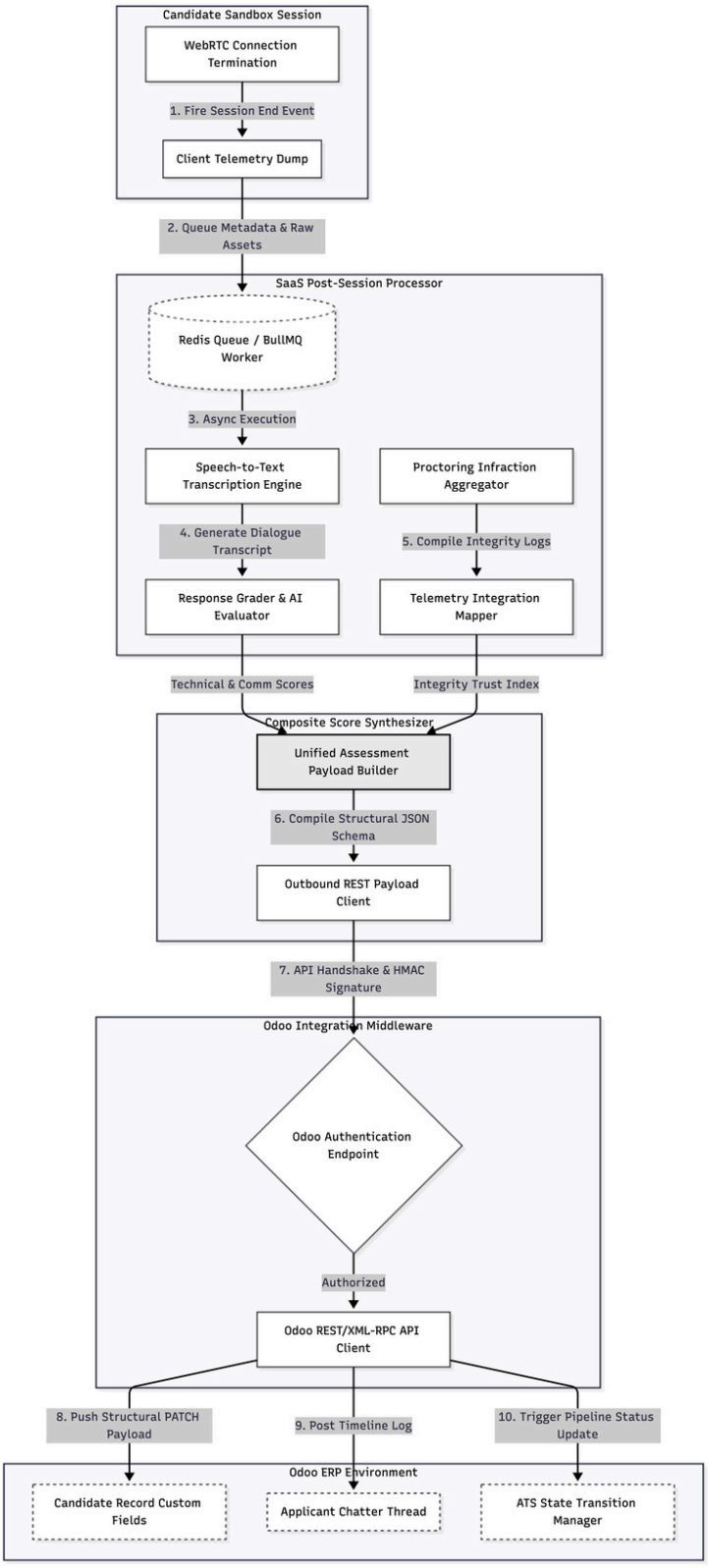
The moment the call ends, the SaaS frontend tears down the WebRTC connection and alerts the backend. Rather than processing metrics synchronously, the session is handed off to an asynchronous worker queue managed by Redis and BullMQ.

The background worker takes the raw audio files and routes them to Deepgram's transcription pipeline, generating a timestamped transcript. This text is passed to an analytical evaluation pipeline powered by Gemini 1.5 Flash, which scores each response against the predefined answer key based on accuracy, clarity, and communication quality.

Simultaneously, the proctoring worker aggregates the client-side telemetry logs. It calculates an **Integrity Trust Index** by checking the count and duration of tab switches, display stream disconnects, and look-away gaze deviations.

The SaaS backend then initiates an outbound connection to the client's Odoo ERP instance via Odoo's external REST/XML-RPC API. The integration engine maps the data directly into Odoo Recruitment fields:

- The candidate's `x_ai_screening_score` and `x_ai_integrity_score` custom fields are updated.
- A detailed narrative summary, complete transcript link, and proctoring logs are appended directly into the applicant's chatter log.
- Automated applicant tags (e.g., `AI-Verified`, `Review-Flagged`) are applied to indicate proctoring results.



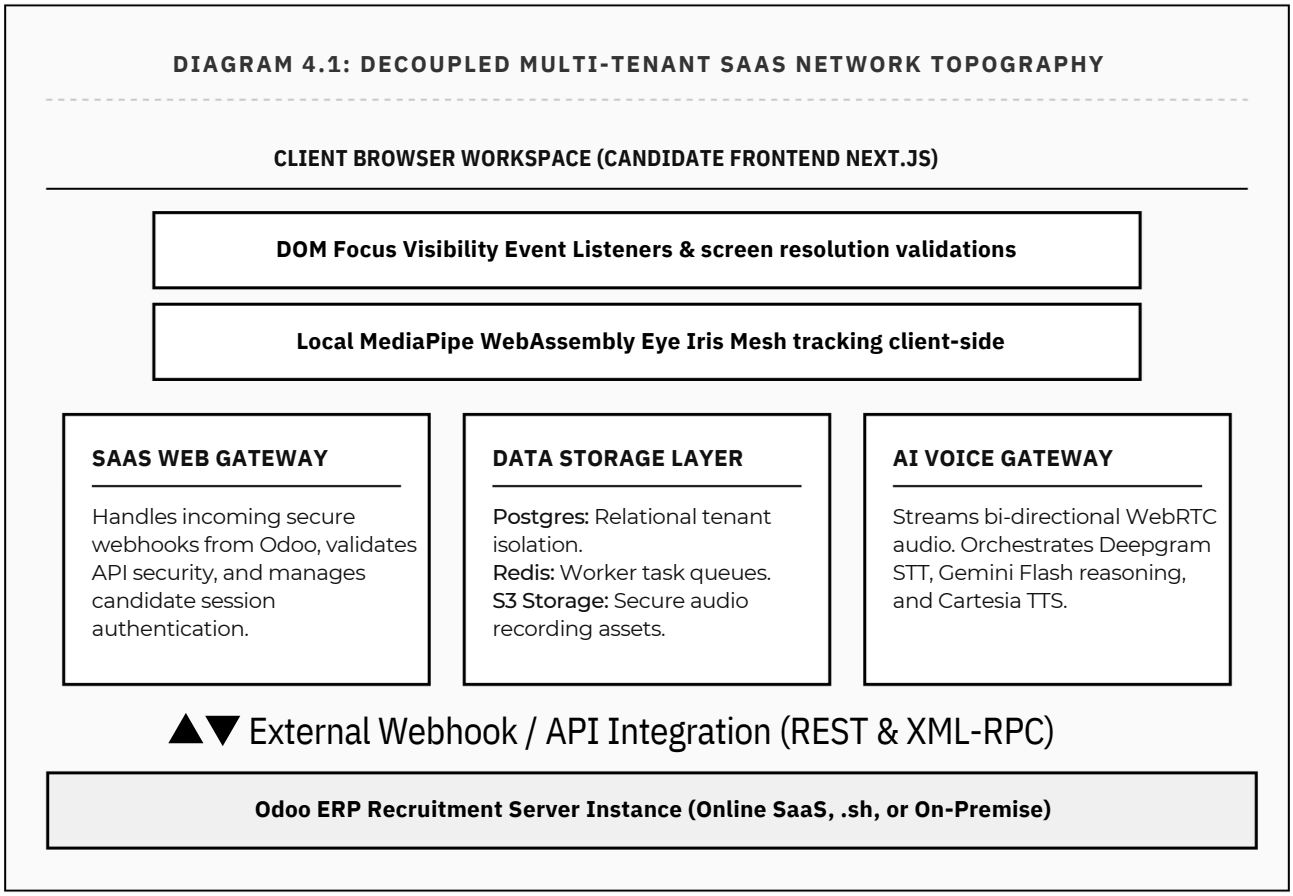
3. Recommended Production Tech Stack Specification

The following architectural grid outlines the production-grade framework specifications, highlighting the strategic engineering decisions driving the platform's deployment.

ARCHITECTURE LAYER	RECOMMENDED FRAMEWORK/ API	STRATEGIC ENGINEERING JUSTIFICATION
Frontend Interface Suite	Next.js 14+ / TypeScript / Tailwind CSS	Provides a robust platform for managing complex application states (permissions wizards, proctoring trackers) with optimal loading speed and minimal client-side overhead.
Client-Side Biometric Vision Engine	Google MediaPipe Face Mesh / WebGazer.js	Runs purely client-side inside the candidate's browser sandbox (WebAssembly-backed). This eliminates the massive server costs and bandwidth requirements of streaming high-definition video feeds to the cloud for real-time analysis.
Backend Application Architecture	FastAPI (Python 3.11+) or NestJS (Node.js)	Both run asynchronous, event-driven development models optimized for processing high-volume HTTP webhook signals and managing concurrent WebSockets or API integrations.
Task Queuing & Memory Caching	Redis Cloud Enterprise + BullMQ	Essential for staging asynchronous background processes. It prevents API gateway timeouts by running heavy audio transcription and grading operations in decoupled background worker threads.
Primary Relational Storage	PostgreSQL (AWS RDS or Supabase)	Essential for multi-tenant SaaS architectures. PostgreSQL provides secure data isolation, stable JSON schema support, and strong transactional integrity for user, tenant, and assessment tables.
Voice Orchestration Gateways	Vapi.ai or Retell AI Core Gateways	Manages low-level WebRTC synchronization, echo cancellation, and Voice Activity Detection (VAD) configurations, reducing development complexity and lowering latency.
Speech-to-Text Subsystem	Deepgram Nova-2 Streaming API	Highly optimized for real-time conversational processing. It transcribes spoken words with sub-200ms latency, providing the speed needed for fluid AI interactions.
Core Language Inference Brain	Google Gemini 1.5 Flash API	Delivers cost-effective reasoning pipelines with ultra-low token generation latencies, maintaining a smooth conversational flow without awkward pauses.

4. Granular Core Architecture Topography

The system topography is designed as an independent SaaS application, communicating with Odoo ERP structures exclusively through external API handshakes.



4.1 The Linguistic Proctoring Fallback System

While standard proctoring metrics track physical movements and browser status, sophisticated candidates may attempt hardware-level workarounds. This includes using hardware-split video feeds, running speech-to-text translators on secondary monitors, or using foot-pedal controllers to fetch external answers without triggering standard browser infractions.

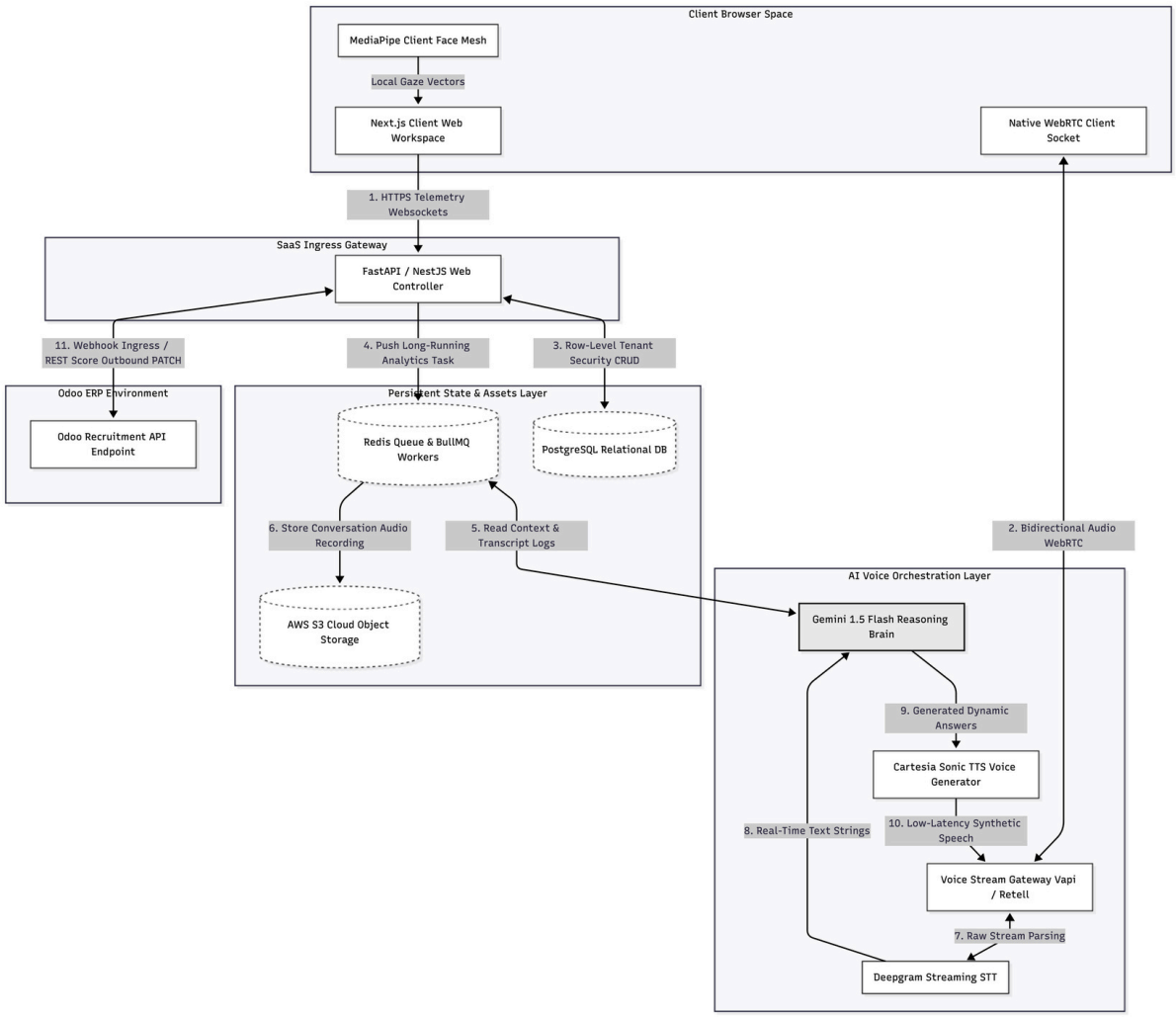
To neutralize these exploits, this platform implements a **Linguistic Proctoring Fallback System**. Operating as an internal validation layer within the evaluation pipeline, this engine analyzes candidate speech patterns:

- **Conversational Latency Monitoring:** The system tracks the exact duration between the end of an AI inquiry and the start of the candidate's response. A constant, unvarying delay (e.g., exactly 4.2 seconds across all responses) indicates the candidate is waiting for external text synthesis tools.

• **Syntactic Complexity vs. Vocal Cadence:** The engine compares the complexity of the spoken words against the candidate's vocal delivery. If a candidate uses complex syntax but speaks with a slow, flat, monotone voice, the system flags the interaction. This pattern typically indicates that the candidate is reading an AI- generated script from an unmonitored screen, rather than speaking dynamically.

• **Dynamic Follow-up Performance:** If a candidate answers a structured difficulty question perfectly but falters, pauses excessively, or contradicts themselves during a dynamic follow-up question, the system flags this performance drop.

These analytical checks detect advanced cheating attempts, automatically assigning a "Suspicious Activity" integrity rating to the Odoo candidate record even if standard browser-level tests pass.



5. Deep Task & Sub-Task Execution Matrix

The following task matrix outlines the modular sub-tasks required to build and deploy each component of the SaaS platform.

Component 1: Odoo Integration Gateway & Question Builder Subsystem

Sub-Task 1.1: Webhook Ingestion Router & Payload Parser
Develop a secure, asynchronous API controller designed to receive HTTPS POST payloads from Odoo when a job vacancy status changes. Implement cryptographic signature validation (HMAC SHA-256) inside the routing middleware using a tenant-specific API secret. The ingestion payload must run in a non-blocking queue (Redis-backed), immediately returning an HTTP 202 Accepted status to the Odoo server to avoid timeouts.

Sub-Task 1.2: Job Requirement Extraction & Normalization
Create a text-processing engine that extracts clean keyword and context structures from unstructured HTML or markdown job descriptions. Implement robust HTML parsing to strip out design noise, CSS selectors, and script blocks. The system must standardize bulleted lists, identify key technical frameworks, database architectures, soft skills, and required experience levels.

Sub-Task 1.3: Stratified Question Generation Matrix
Build an LLM-driven query pipeline that generates a multi-tiered interview question framework based on normalized job requirements. Design system prompts using Gemini 1.5 Flash to enforce structured outputs via JSON schemas. The model must return exactly 4 difficulty tiers (Basic, Easy, Medium, Hard) containing target questions, expected answer highlights, and specific evaluation metrics.

Sub-Task 1.4: Multi-Tenant Data Staging Layer
Design PostgreSQL database schemas to store, organize, and protect tenant and question matrix records. Implement Row-Level Security (RLS) within PostgreSQL to ensure strict data isolation between corporate tenants. Optimize database indices across Tenant ID, Job ID, and Candidate ID fields to ensure sub-millisecond query responses during active sessions.

Component 2: Frontend Candidate Portal & Hardened Proctoring Suite

Sub-Task 2.1: Device Access & Diagnostics Validation Layer
Build an interactive onboarding wizard that checks hardware readiness and permissions before granting access to the interview. Write client-side checks to verify microphone and camera availability. The interface must block access if devices are unavailable, testing audio input thresholds to ensure clear speech recording.

Sub-Task 2.2: True Full-Screen Enforcement Logic
Create a client-side screen-sharing mechanism that requires candidates to share their entire desktop. Use the browser's native getDisplayMedia() API. The system must analyze the media track properties: if the stream width and height do not match the system's screen resolution (indicating a shared tab or window), the interface must block candidate progression and require a full-screen share.

Sub-Task 2.3: Active Window Focus Monitor
Build a continuous browser tracking system to detect when a candidate navigates away from the interview interface. Implement event tracking linked to the HTML5 Page Visibility API (visibilitychange), monitoring window.onblur and window.onfocus states. The system must immediately log any tab-switching or window defocus events with accurate microsecond timestamps.

Sub-Task 2.4: Vision-Based Gaze Deviation Monitor
Implement real-time client-side gaze and face tracking to detect when a candidate looks off-screen. Integrate a client-side eye-tracking library (WebGazer.js) compiling down to WebAssembly. The browser must analyze facial landmarks locally, logging visual deviations if the candidate's gaze drifts from the center screen for more than 3 consecutive seconds during active question phases.

Component 3: Live Conversational Voice AI Streaming Engine

Sub-Task 3.1: Voice Service WebRTC Client Integration
Connect the client frontend to the voice orchestration layer (Vapi/Retell) via WebRTC to enable smooth, real-time voice interactions. Implement clean WebRTC signaling protocols using JSON web socket exchanges. The connection must handle network transitions, monitor call quality metrics, and provide fallback audio controls to ensure call continuity.

Sub-Task 3.2: Context Injection & Session Bootstrapping
Create a session orchestration system that retrieves the pre-generated question matrix from PostgreSQL the moment a candidate connects, transforming those records into instructions passed directly to the streaming AI runtime agent. Securely fetch session-specific question sets from PostgreSQL.

Sub-Task 3.3: Dynamic Follow-Up Inquiry Logic
Configure prompt engineering guidelines to control the AI's speaking behavior during live interactions. The model is trained to listen to candidate text, evaluate its depth, and dynamically ask exactly 1 to 2 follow-up questions to check for genuine comprehension before moving to the next pre-set difficulty tier.

Sub-Task 3.4: Interruption Handling & Silences Configuration Tuning
Fine-tune the conversational flow to ensure natural turn-taking and allow candidates to interrupt the AI mid-sentence. Configure Voice Activity Detection (VAD) latency thresholds. Adjust settings to ensure the AI stops speaking immediately if the user speaks (interruption/barge-in), while ignoring short pauses when a candidate is thinking.

Component 4: Evaluation Aggregation, Analytics & Feedback Pipeline

Sub-Task 4.1: Asynchronous Assessment Background Worker
Deploy an automated background task process (Redis/BullMQ) that runs the moment a candidate disconnects. The service verifies the complete delivery of text transcript files, raw call metrics, and audio asset links before running analytical tasks.

Sub-Task 4.2: Comprehensive Score Evaluation Logic
Pass the compiled transcript text back to the primary AI reasoning engine to measure candidate response depth against the original job evaluation parameters, returning a normalized technical competency rating out of 100.

Sub-Task 4.3: Proctoring Telemetry Anomaly Aggregation
Aggregate all front-end proctoring events—including tab switch durations, full-screen violations, and visual look-away infractions—into a unified candidate trust metric profile. Calculate a normalized Integrity Score, automatically marking profiles with high infractions for recruiter review.

Sub-Task 4.4: Odoo API Outbound Integration Engine
Build an outbound update client utilizing external REST/XML-RPC connection models. This service patches relevant records directly inside Odoo, instantly inserting candidate evaluation scores, summary narratives, and proctoring links right into the recruiter's applicant tracking dashboard.

6. Security, Privacy & Technical Risks Mitigation Matrix

Operating an automated vetting and proctoring platform requires strict security controls. The following matrix details core technical vulnerabilities and architectural mitigation strategies.

IDENTIFIED RISK FACTOR	TECHNICAL THREAT DESCRIPTION	SAAS ARCHITECTURAL MITIGATION STRATEGY
Data Privacy & Compliance (GDPR/ CCPA)	Capturing webcam feeds, iris coordinate tracking, and screen recordings constitutes the collection of sensitive biometric data, risking compliance failures.	Ensure the system runs without sending raw video feeds to the cloud. All face mesh and gaze analyses are processed locally in the browser, with the webcam feed kept strictly in memory and never written to disk. The candidate must provide explicit, cookie-style consent before permissions are requested. Implement automated, database-driven purging of transcripts and call audios 30 days post-evaluation.
Mid-Call Session Disconnects	Network dropouts, browser crashes, or API gateway failures can disrupt high-stakes candidate sessions midway through.	Implement local state caching inside the Next.js client app. If the WebRTC connection drops, the frontend displays a localized modal explaining that it is reconnecting. Background workers can resume the interview from the last completed difficulty tier once the connection is restored, preventing candidates from having to repeat already-completed sections.
Virtual Audio Cable Exploitation	Candidates can route system audio through virtual pipelines to automate transcription, feeding the text to another device to generate answers.	Use the Linguistic Proctoring Fallback System. The evaluation engine monitors candidate speech pattern characteristics, assessing conversational response latency, vocal delivery cadence, and performance drops during dynamic follow-up questions. If a candidate reads a generated script, the system flags the profile for review regardless of whether hardware infractions were detected.
Hardware Access Inequities	Legitimate applicants using older hardware or working in low-light environments may trigger false look-away flags.	Design the proctoring engine to evaluate overall behavioral trends rather than relying on binary alert triggers. The eye-tracking system establishes a continuous calibration curve throughout the interview, adjusting to candidate movements. Recruiters can adjust proctoring sensitivity sliders inside the SaaS settings panel to account for variable candidate environments.